

Tuyển Tập Bài Tập: Cây Phân Đoạn & Kỹ Thuật Đánh Dấu Trễ (Segment Tree & Lazy Propagation)

(Curated Competitive Programming Problems on Segment Tree, Lazy Propagation & Advanced Tagging)

Tác giả: Duc-Minh Vu | **Đơn vị:** SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA), Đại học Kinh tế Quốc dân (NEU)

Soạn thảo: Được hỗ trợ và soạn thảo bởi Agentic AI tool

Bản quyền © 2026 **Duc-Minh Vu**. Mọi quyền được bảo lưu. Tài liệu đào tạo và bồi dưỡng sinh viên Olympic Tin học / ICPC.

Bảng Tổng Hợp 16 Bài Tập Tuyển Chọn

STT	Mã Bài	Tên Bài Toán	Nguồn	Độ Khó	Chủ Đề Trọng Tâm
1	CSES 1648	Dynamic Range Sum Queries	CSES	☆☆☆	PURQ Sum chuẩn mực, cập nhật điểm và truy vấn tổng
2	CSES 1649	Dynamic Range Minimum Queries	CSES	☆☆☆	PURQ RMQ động, cập nhật điểm và tìm min đoạn
3	CSES 1651	Range Update Queries	CSES	☆☆☆	RUPQ cơ bản với Lazy Propagation (cộng đoạn, hỏi điểm)
4	CSES 1650	Range Xor Queries	CSES	☆☆☆	Phép toán XOR trên cây phân đoạn
5	CSES 1735	Range Updates and Sums	CSES	☆☆☆☆	Lazy kép: Kết hợp phép gán đè và phép cộng dồn ($ax + b$)
6	CSES 1736	Polynomial Queries	CSES	☆☆☆☆	Cộng cấp số cộng $A[i] += (i - l + 1)$ vào đoạn $[l, r]$
7	CF 292E	Copying Data	Codeforces	☆☆☆☆	Lazy Propagation ghi nhận mốc thời gian copy đoạn $A \rightarrow B$
8	CF 339D	Xenia and Bitwise Operations	Codeforces	☆☆☆☆	Segment Tree xen kẽ phép OR và XOR theo tầng chẵn/lẻ
9	CF 438D	The Child and Sequence	Codeforces	☆☆☆☆	Phân rã Modulo đoạn $A[i] = A[i] \bmod x$ dựa trên Max đoạn
10	CSES 1190	Subarray Sum Queries	CSES	☆☆☆☆	Maximum Subarray Sum động qua 4 trường thông tin Node

STT	Mã Bài	Tên Bài Toán	Nguồn	Độ Khó	Chủ Đề Trọng Tâm
11	CF 522D	Closest Equals	Codeforces	★★★★★	Quét Offline kết hợp Segment Tree RMQ khoảng cách 2 số bằng nhau
12	CSES 2166	Prefix Sum Queries	CSES	★★★★★	Truy vấn tiền tố lớn nhất trên đoạn con động $O(\log N)$
13	CF 1405E	Fixed Point Removal	Codeforces	★★★★★	Walk on Segment Tree tìm vị trí xóa điểm bất động
14	CF 1439C	Greedy Shopping	Codeforces	★★★★★	Mô phỏng mua sắm tham lam bằng Walk on Tree + Lazy Cholesky
15	SPOJ GSS2	Can you answer these queries II	SPOJ	★★★★★	Tổng đoạn con không trùng lặp, Historical Maximum Lazy Tag
16	CF 1290E	Cartesian Tree	Codeforces	★★★★★	Segment Tree Beats (Chinos Segment Tree) duy trì kích thước cây

💡 Phân Tích & Hướng Dẫn Chi Tiết (3-Tier Hints)

Bài 1: CSES 1648 — Dynamic Range Sum Queries

- **Đề bài:** Cho mảng A gồm N phần tử ($N, Q \leq 2 \times 10^5$). Thực hiện 2 loại thao tác: (1) Cập nhật giá trị $A[k] = u$, (2) Tính tổng đoạn $\sum_{i=a}^b A[i]$.
- **Gợi ý 1 (Nhận xét bản chất):**
 - Bài toán kinh điển nhất của Cây Phân Đoạn cập nhật điểm (PURQ).
- **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Xây dựng Segment Tree quản lý mảng với N lá. Mỗi nút lưu trường `sum`.
 - Hàm `pull(id): tree[id] = tree[2 * id] + tree[2 * id + 1]`.
 - Cập nhật điểm $O(\log N)$, truy vấn tổng $O(\log N)$.
- **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
 - Thời gian: $O(N + Q \log N)$, Bộ nhớ: $O(4N)$.
 - Cạm bẫy: Tổng mảng có thể lên tới 2×10^{14} , bắt buộc dùng kiểu `long long` cho trường `sum`.

Bài 2: CSES 1649 — Dynamic Range Minimum Queries

- **Đề bài:** Cho mảng A gồm N số nguyên ($N, Q \leq 2 \times 10^5$). Thực hiện 2 loại thao tác: (1) Cập nhật $A[k] = u$, (2) Tìm giá trị nhỏ nhất $\min_{i=a}^b A[i]$.
- **Gợi ý 1 (Nhận xét bản chất):**
 - Cây Phân Đoạn hỗ trợ tìm Min động cực kỳ linh hoạt mà Sparse Table không thể làm được do có thao tác cập nhật.
- **Gợi ý 2 (Thuật toán & Tối ưu):**

- Mỗi nút lưu giá trị `min_val`.
- Hàm `pull(id)`: `tree[id] = std::min(tree[2 * id], tree[2 * id + 1])`.
- Giá trị trung hòa khi đoạn truy vấn rời nhau là $+\infty$ (`2e9` hoặc `2e18`).
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N + Q \log N)$, Bộ nhớ: $O(4N)$.
- Cận bậc: Khởi tạo giá trị vô cùng đủ lớn để tránh so sánh sai.

Bài 3: CSES 1651 — Range Update Queries

- **Đề bài:** Cho mảng A gồm N phần tử. Hỗ trợ 2 thao tác: (1) Cộng thêm u vào tất cả phần tử trong đoạn $[a, b]$, (2) Truy vấn giá trị hiện tại của phần tử $A[k]$.
- **Gợi ý 1 (Nhận xét bản chất):**
- Thao tác Range Update - Point Query (RUPQ).
- Có thể giải bằng Fenwick Tree trên mảng hiệu hoặc Segment Tree có Lazy Propagation.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Dùng Segment Tree: Mỗi nút lưu `val` và `lazy`.
- Khi cập nhật đoạn $[a, b]$, nếu đoạn $[l, r]$ nằm trọn trong $[a, b]$, ta chỉ cộng `lazy[id] += u` và return ngay.
- Khi truy vấn vị trí k , ta đi từ gốc xuống lá k , cộng dồn tất cả các giá trị `lazy` trên đường đi.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O((N + Q) \log N)$, Bộ nhớ: $O(4N)$.

Bài 4: CSES 1650 — Range Xor Queries

- **Đề bài:** Cho mảng A gồm N phần tử ($N, Q \leq 2 \times 10^5$). Tìm giá trị $A_a \oplus A_{a+1} \oplus \dots \oplus A_b$.
- **Gợi ý 1 (Nhận xét bản chất):**
- Phép toán XOR có tính chất tự nghịch đảo: $x \oplus x = 0$.
- Bài toán này có thể giải trong $O(1)$ bằng mảng tiền tố XOR (Prefix XOR).
- Tuy nhiên, nếu bài toán có thêm cập nhật điểm ($A[k] = u$), ta dùng Segment Tree với phép kết hợp là XOR: `tree[id] = tree[2*id] ^ tree[2*id+1]`.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Phần tử trung hòa của phép XOR là 0 (vì $x \oplus 0 = x$).
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(N + Q \log N)$, Bộ nhớ: $O(4N)$.

Bài 5: CSES 1735 — Range Updates and Sums

- **Đề bài:** Cho mảng A . Thực hiện 3 loại truy vấn:
- Cộng thêm x vào đoạn $[a, b]$.
- Gán tất cả phần tử trong đoạn $[a, b]$ bằng x .
- Tính tổng các phần tử trong đoạn $[a, b]$.
- **Gợi ý 1 (Nhận xét bản chất):**

- Bài toán kinh điển về Lazy kép: Phép gán đè làm mất hiệu lực của các phép cộng dồn trước đó!
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Mỗi nút lưu nhãn gán `set_val` (mặc định -1) và nhãn cộng `add` (mặc định 0).
- Khi nhận thao tác gán x : `set_val = x`, xóa `add = 0`.
- Khi nhận thao tác cộng x : Nếu nút đang có `set_val != -1`, ta cộng thẳng vào `set_val += x`; ngược lại cộng vào `add += x`.
- Trong hàm `push`, truyền `set_val` trước, sau đó truyền `add`.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(Q \log N)$, Bộ nhớ: $O(4N)$.
- Cận bậc: Thứ tự ưu tiên giữa gán và cộng phải được xử lý chặt chẽ như mô hình affine $f(x) = a \cdot x + b$.

Bài 6: CSES 1736 — Polynomial Queries

- **Đề bài:** Cho mảng A . Hỗ trợ 2 thao tác:
- Với đoạn $[a, b]$, cộng thêm 1 vào $A[a]$, cộng 2 vào $A[a + 1], \dots$, cộng $(b - a + 1)$ vào $A[b]$.
- Tính tổng các phần tử trong đoạn $[a, b]$.
- **Gợi ý 1 (Nhận xét bản chất):**
- Giá trị cộng vào vị trí $i \in [a, b]$ có dạng cấp số cộng: $\Delta A_i = (i - a + 1) = i - (a - 1)$.
- Phân tích thành 2 thành phần độc lập: Phép cộng $i \cdot c$ và phép cộng hằng số d .
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Mỗi nút của Segment Tree quản lý đoạn $[l, r]$ lưu hai nhãn lười:
 - `cnt`: Số lần đoạn này được cộng cấp số cộng bắt đầu từ l .
 - `step`: Công sai cộng dồn.
- Tổng cấp số cộng trên đoạn độ dài $len = r - l + 1$ với số hạng đầu u_1 và công sai $d = 1$:

$$S = \frac{len \cdot (2u_1 + (len - 1) \cdot 1)}{2}$$

- Khi chia đôi đoạn tại mid , nửa trái nhận số hạng đầu u_1 , nửa phải nhận số hạng đầu $u_1 + (mid - l + 1)$.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(Q \log N)$, Bộ nhớ: $O(4N)$.
- Cận bậc: Cần thận trọng số khi nhân độ dài đoạn với cấp số cộng.

Bài 7: Codeforces 292E — Copying Data

- **Đề bài:** Cho hai mảng A và B độ dài N ($N, Q \leq 10^5$). Có 2 loại truy vấn:
- Sao chép đoạn k phần tử từ $A[x \dots x + k - 1]$ đè lên $B[y \dots y + k - 1]$.
- Truy vấn giá trị hiện tại của $B[p]$.
- **Gợi ý 1 (Nhận xét bản chất):**
- Ta không cần thực sự copy từng phần tử.
- Giá trị của $B[p]$ được quyết định bởi **thao tác copy cuối cùng** phủ lên vị trí p .
- **Gợi ý 2 (Thuật toán & Tối ưu):**

- Dựng Segment Tree trên mảng B . Mỗi nút lưu nhãn `lazy` là chỉ số của thao tác copy gần nhất phủ lên đoạn đó.
- Khi có truy vấn loại 1 thứ qid : Gán nhãn `lazy[id] = qid` trên đoạn $[y, y + k - 1]$.
- Khi truy vấn vị trí p : Đi từ gốc xuống lá p , lấy nhãn `qid` lớn nhất trên đường đi.
- Nếu không có thao tác nào phủ: Trả về $B[p]$ ban đầu. Nếu có thao tác qid (copy $A[x \dots]$ sang $B[y \dots]$): Trả về $A[x + (p - y)]$.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(Q \log N)$, Bộ nhớ: $O(4N)$.

Bài 8: Codeforces 339D — Xenia and Bitwise Operations

- **Đề bài:** Cho mảng có 2^n phần tử ($n \leq 17$). Cây tính toán xen kẽ: Tầng 1 tính $A[1] \mid A[2], A[3] \mid A[4], \dots$; Tầng 2 tính XOR của các kết quả tầng 1; Tầng 3 lại tính OR, cứ thế xen kẽ cho tới gốc. Hỗ trợ M truy vấn cập nhật điểm $A[p] = b$ và in ra giá trị tại gốc sau mỗi cập nhật.
- **Gợi ý 1 (Nhận xét bản chất):**
- Cấu trúc tính toán của đề bài chính là một Segment Tree hoàn chỉnh với đúng 2^n lá và chiều cao n .
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Gán nhãn cho mỗi tầng của cây: Tầng lá có khoảng cách đến lá bằng 0.
- Nếu khoảng cách đến lá là số lẻ: Nút này thực hiện phép toán OR (`|`).
- Nếu khoảng cách đến lá là số chẵn: Nút này thực hiện phép toán XOR (`^`).
- Mỗi cập nhật điểm đi từ lá lên gốc cập nhật lại trong $O(n) = O(\log(2^n))$.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**
- Thời gian: $O(2^n + M \cdot n)$, Bộ nhớ: $O(2^{n+2})$.

Bài 9: Codeforces 438D — The Child and Sequence

- **Đề bài:** Cho mảng A gồm N số ($N, Q \leq 10^5$). Hỗ trợ 3 loại truy vấn:
- Tính tổng đoạn $\sum_{i=l}^r A[i]$.
- Lấy modulo đoạn: $A[i] = A[i] \bmod x$ với mọi $i \in [l, r]$.
- Cập nhật điểm: $A[k] = x$.
- **Gợi ý 1 (Nhận xét bản chất):**
- Phép toán modulo không có tính chất phân phối trên phép cộng và không thể truyền nhãn Lazy đơn giản.
- Tuy nhiên: Nếu $A[i] < x$, thì $A[i] \bmod x = A[i]$ (không đổi!).
- Nếu $A[i] \geq x$, thì $A[i] \bmod x < A[i]/2$ (giá trị giảm đi ít nhất một nửa!).
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Mỗi nút lưu `sum` và `max_val`.
- Khi thực hiện truy vấn modulo x trên đoạn $[l, r]$:
 - Nếu `max_val` < x : Đoạn này không có phần tử nào thay đổi, return ngay!
 - Nếu `max_val` $\geq x$: đệ quy xuống các con. Khi chạm nút lá, $A[l] = A[l] \bmod x$.
- Vì mỗi phần tử chỉ bị giảm tối đa $\log_2(\max A)$ lần trước khi về 0, tổng thời gian của tất cả các thao tác modulo được chia đều (Amortized) là $O(N \log N \log(\max A))$.
- **Gợi ý 3 (Độ phức tạp & Cận bậc):**

- Thời gian: $O((N + Q) \log N \log(\max A))$, Bộ nhớ: $O(4N)$.

Bài 10: CSES 1190 — Subarray Sum Queries

- **Đề bài:** Cho mảng A gồm N phần tử ($N, Q \leq 2 \times 10^5$). Có Q truy vấn cập nhật điểm $A[k] = x$. Sau mỗi truy vấn, tìm tổng lớn nhất của một đoạn con liên tiếp bất kỳ (có thể rỗng, tổng ≥ 0).
- **Gợi ý 1 (Nhận xét bản chất):**
- Đây là phiên bản động của bài toán Maximum Subarray Sum (Thuật toán Kadane).
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Mỗi nút của Segment Tree cần lưu 4 trường thông tin:
 1. `sum`: Tổng tất cả các phần tử trong đoạn.
 2. `pref`: Tổng tiền tố lớn nhất của đoạn (≥ 0).
 3. `suff`: Tổng hậu tố lớn nhất của đoạn (≥ 0).
 4. `ans`: Tổng đoạn con liên tiếp lớn nhất trong đoạn (≥ 0).
- Công thức `combine` giữa con trái L và con phải R :
 - `sum = L.sum + R.sum`
 - `pref = std::max(L.pref, L.sum + R.pref)`
 - `suff = std::max(R.suff, R.sum + L.suff)`
 - `ans = std::max({L.ans, R.ans, L.suff + R.pref})`
- **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
- Thời gian: $O(N + Q \log N)$, Bộ nhớ: $O(4N)$.
- Cạm bẫy: Trường hợp rỗng được phép lấy tổng bằng 0.

Bài 11: Codeforces 522D — Closest Equals

- **Đề bài:** Cho mảng A gồm N phần tử ($N, Q \leq 5 \times 10^5$). Cho Q truy vấn $[L, R]$. Tìm khoảng cách nhỏ nhất giữa hai vị trí có cùng giá trị: $\min\{y - x \mid L \leq x < y \leq R, A[x] = A[y]\}$. Nếu không có, in -1.
- **Gợi ý 1 (Nhận xét bản chất):**
- Bài toán xử lý Offline: Sắp xếp các truy vấn theo cận phải R tăng dần.
- Đặt $prev[i]$ là vị trí xuất hiện gần nhất của giá trị $A[i]$ ở bên trái ($prev[i] < i$).
- Khi xét tới vị trí R , nếu $prev[R]$ tồn tại, ta có một cặp hợp lệ tại $x = prev[R]$ với khoảng cách $R - prev[R]$.
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Dựng Segment Tree RMQ trên các vị trí $x \in [1, N]$.
- Khi duyệt R từ 1 đến N : Nếu tồn tại $p = prev[R]$, cập nhật tại vị trí p : `st.update(p, R - p)`.
- Với các truy vấn có cận phải bằng R , đáp án là `st.query(L, R)`.
- **Gợi ý 3 (Độ phức tạp & Cạm bẫy):**
- Thời gian: $O((N + Q) \log N)$, Bộ nhớ: $O(N + Q)$.

Bài 12: CSES 2166 — Prefix Sum Queries

- **Đề bài:** Cho mảng A gồm N phần tử ($N, Q \leq 2 \times 10^5$). Hỗ trợ 2 thao tác: (1) Cập nhật $A[k] = u$, (2) Tìm giá trị tiền tố lớn nhất của đoạn $[a, b]$, tức $\max_{a-1 \leq k \leq b} \sum_{i=a}^k A[i]$ (cho phép tiền tố rỗng với tổng = 0).
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Tương tự như bài toán Subarray Sum, nhưng ta chỉ cần duy trì `sum` và `pref`.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Nút lưu `sum` và `pref = max(0LL, max_prefix_sum)`.
 - Công thức gộp: `sum = L.sum + R.sum`, `pref = std::max(L.pref, L.sum + R.pref)`.
 - Truy vấn trên đoạn $[a, b]$ trả về `pref`.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(N + Q \log N)$, Bộ nhớ: $O(4N)$.
-

Bài 13: Codeforces 1405E — Fixed Point Removal

- **Đề bài:** Mảng A có phần tử được gọi là điểm bất động nếu $A[i] = i$. Mỗi thao tác được xóa một điểm bất động, các phần tử phía sau dồn lên trước. Cho Q truy vấn: Nếu xóa x phần tử đầu và y phần tử cuối, số điểm bất động tối đa có thể xóa là bao nhiêu?
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Một phần tử $A[i]$ cần bị giảm $i - A[i]$ đơn vị để trở thành điểm bất động.
 - Điều này đòi hỏi phải có ít nhất $i - A[i]$ phần tử đứng trước nó được xóa.
 - Nếu $A[i] > i$, phần tử này không bao giờ xóa được.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Xử lý Offline theo cận phải i .
 - Dùng **Walk on Segment Tree** để tìm vị trí L xa nhất sang phải sao cho đoạn $[L, i]$ có đủ số lượng phần tử xóa trước đó $\geq i - A[i]$.
 - Sau đó cập nhật cộng 1 vào tiền tố $[1, L]$ bằng Lazy Segment Tree.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O((N + Q) \log N)$, Bộ nhớ: $O(4N)$.
-

Bài 14: Codeforces 1439C — Greedy Shopping

- **Đề bài:** Cho dãy A không tăng ($A_1 \geq A_2 \geq \dots \geq A_N$). Có 2 truy vấn:
- Với đoạn $[1, x]$, cập nhật $A[i] = \max(A[i], y)$.
- Bắt đầu từ vị trí x với số tiền y , đi từ trái sang phải, cứ gặp món hàng nào mua được ($A[i] \leq \text{tiền hiện tại}$) thì mua và trừ tiền. Đếm số món mua được.
- **Gợi ý 1 (Nhận xét bản chất):**
- Mảng luôn duy trì tính chất không tăng sau mọi thao tác loại 1 (Cholesky-like update).
- Thao tác 1 tương đương với tìm vị trí đầu tiên có $A[i] < y$ bằng Walk on Tree, sau đó gán đoạn bằng y .
- **Gợi ý 2 (Thuật toán & Tối ưu):**
- Trong truy vấn 2: Dùng Walk on Tree tìm vị trí đầu tiên $\geq x$ có giá trị $\leq y$.

- Sau đó dùng Walk on Tree tìm đoạn con dài nhất từ vị trí đó có tổng $\leq y$, mua toàn bộ đoạn đó và trừ tiền.
 - Lặp lại quá trình này. Vì mỗi bước số tiền giảm ít nhất một nửa, ta chỉ lặp tối đa $O(\log(\text{tiền}))$ lần!
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(Q \log N \log(\max Y))$, Bộ nhớ: $O(4N)$.
-

Bài 15: SPOJ GSS2 — Can you answer these queries II

- **Đề bài:** Cho mảng A ($N, Q \leq 10^5$). Truy vấn $[l, r]$ tìm tổng đoạn con liên tiếp lớn nhất sao cho **mỗi giá trị xuất hiện nhiều lần chỉ được tính đúng 1 lần**.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Xử lý Offline theo cận phải R .
 - Khi duyệt tới R , giá trị $A[R]$ chỉ đóng góp vào các đoạn con bắt đầu từ $(prev[A[R]] + 1)$ đến R .
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Thao tác cộng đoạn $A[R]$ vào khoảng $[prev[A[R]] + 1, R]$.
 - Ta cần tìm giá trị lớn nhất trong lịch sử (Historical Maximum) của các nút trên cây phân đoạn!
 - Cần duy trì nhãn lịch sử: `lazy_add` và `max_lazy_add` trong suốt thời gian tồn tại.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O((N + Q) \log N)$, Bộ nhớ: $O(4N)$.
 - Cận bậc: Kỹ thuật nhãn lịch sử đòi hỏi công thức hợp nhất nhãn cực kỳ chuẩn xác.
-

Bài 16: Codeforces 1290E — Cartesian Tree

- **Đề bài:** Cho một hoán vị. Lần lượt chèn các số từ $1 \dots N$ vào mảng. Sau mỗi lần chèn, tính tổng kích thước các cây con trong Cartesian Tree của mảng hiện tại.
 - **Gợi ý 1 (Nhận xét bản chất):**
 - Kích thước cây con của đỉnh i trong Cartesian Tree là $R_i - L_i + 1$, với L_i là vị trí phần tử lớn hơn đầu tiên bên trái, và R_i là vị trí phần tử lớn hơn đầu tiên bên phải.
 - Khi chèn một phần tử mới lớn hơn tất cả phần tử trước đó, nó sẽ cắt đứt các đoạn bao phủ của các phần tử cũ.
 - **Gợi ý 2 (Thuật toán & Tối ưu):**
 - Sử dụng cấu trúc **Segment Tree Beats (Ji Driver / Chinos Segment Tree)**.
 - Mỗi nút duy trì giá trị lớn nhất `max1`, giá trị lớn nhì `max2`, và số lượng phần tử đạt `max1`.
 - Hỗ trợ thao tác cập nhật $A[i] = \min(A[i], x)$ trong $O(\log N)$ amortized.
 - **Gợi ý 3 (Độ phức tạp & Cận bậc):**
 - Thời gian: $O(N \log^2 N)$ hoặc $O(N \log N)$.
 - Cận bậc: Cài đặt Segment Tree Beats đòi hỏi theo dõi chặt chẽ điều kiện dừng đệ quy.
-



Competitive Programming Handbook

SLSCM Lab — Faculty of Data Science and Artificial Intelligence (FDA) — National Economics University (NEU)

Tài liệu được soạn thảo và tối ưu bởi Agentic AI tool

© 2026 Duc-Minh Vu. Toàn bộ bản quyền được bảo lưu.